



µLab worksheet • Deploying Spark on your own cluster

Group _____ Date _____ Time _____

Driver (types) _____ Mapper (looks up) _____ Referee (demos) _____

Hand this in at the end of the round. The referee presents this work in 90 seconds at the caucus, so the referee has to follow it, not just watch it.

1 • RE-CREATE THE CLUSTER WITH THE LAB FOLDER ATTACHED • STEP 1

µLab 3 will run a word count over 18'846 files that live in this repository. The cluster from µLab 1 cannot read any of them, and no command will fix that: a container can only see the folders it was handed **at the moment it was created**, and that command said nothing about this repository. So the cluster is thrown away and made again, this time with the folder attached.

1 ☐ Get the repository, and work from inside it.

Every command on this sheet is typed on your laptop, from this folder. If the clone is already there from the announcement, `git pull` is enough.

```
git clone https://github.com/oesteban/isc-302-2026-2027-spark-lab
cd isc-302-2026-2027-spark-lab
```

clone

`https://github.com/...`

Copies the whole repository, history included, into a new folder named after it.

The address. 17.6 MiB to download, which is quick; then 18'846 small files to write to disk, which is not. A checkout that takes a minute is working, not hung.

Now replace the cluster.

The first line deletes the µLab 1 cluster and everything on it, including the whale. That is intended: nothing from µLab 1 is needed again.

```
docker rm -f k8s
docker run -d --name k8s --privileged --tmpfs /run --tmpfs /var/run \
  -p 6443:6443 -v "$PWD/kube:/output" -v "$PWD:/lab" \
  rancher/k3s:v1.36.4-k3s1 server --disable=traefik \
  --write-kubeconfig /output/config --write-kubeconfig-mode 644
```

This is the µLab 1 command with **one argument added**. The table repeats the rest in one line each so that nothing on this sheet is typed unexplained.

```
rm -f k8s
-d
--name k8s
--privileged
--tmpfs /run
--tmpfs /var/run
-p 6443:6443
-v "$PWD/kube:/output"
-v "$PWD:/lab"
```

Delete the container named `k8s`. The `-f` kills it first if it is still running.

Detached: the cluster keeps running after your prompt comes back.

Give it a name, so later commands can say `k8s` instead of an id.

Kubernetes runs containers, so it needs the kernel privileges a container is normally denied.

A small in-memory filesystem. `k3s` writes sockets there and they must not survive a restart.

The same, for the second conventional location.

Publish the API port to your laptop, so a client outside the container can reach it.

Bind-mount: the `kube` folder here appears inside at `/output`, which is where the credentials file gets written.

The new one. This whole folder, corpus included, appears inside the cluster container at `/lab`. Pods will mount it from there in step 3.

```
rancher/k3s:...
server
--disable=traefik
--write-kubeconfig ...
```

The image: a whole Kubernetes in one binary. Pinned to `v1.36.4-k3s1` so every laptop runs the same version.

Everything after the image name is handed to the program inside. `server` means be the cluster, not join one.

Skip the bundled web router. Nothing here serves web traffic, and it takes time to start.

Where to write the credentials file, and with which permissions. `644` makes it readable by you rather than only by root.

Then get a client, and point it at the folder too.

Linux users who installed `kubectl` in µLab 1 keep using it and skip this box. Everyone else runs the client in its own container, as in µLab 1, with **two changes**: the mount is now the whole folder rather than just `kube`, so that the client can read the file applied in step 3.

```
docker run --rm -it --network container:k8s \
  -v "$PWD:/lab:ro" -e KUBECONFIG=/lab/kube/config \
  --entrypoint sh alpine/kubectl
```

`--rm -it`

Disposable, and interactive: a shell you can type many commands into, deleted when you leave.

```
--network container:k8s
-v "$PWD:/lab:ro"
-e KUBECONFIG=/lab/kube/config
--entrypoint sh
alpine/kubectl
```

Join the cluster container's network instead of getting one. That is why `127.0.0.1:6443` works in here. **Changed.** `µLab 1` mounted only `kube`. The client now needs the manifest file as well, so the whole folder comes in, read-only.

Changed to match. The credentials are at the same place as before, one level further down the new mount.

Run a shell instead of the single `kubectl` the image is built for. The image: `kubectl` and a small Alpine base, from Docker Hub.

Check the cluster answers. The `NAME` is the new container's id, so it is **not** the one `µLab 1` showed: this is a different cluster.

```
kubectl get nodes
```

`NAME` _____ `STATUS` _____

2 • SOME DEFINITIONS

Everything this round asks you to explain can be answered from this block. It is worth reading once before step 2 rather than hunting through it afterwards.

image store	The place a container runtime keeps the images it can start. Your laptop's Docker has one; the cluster has a separate one of its own, and neither can read the other's. <code>µLab 1</code> showed this by listing them side by side. Step 2 is what to do about it.
driver	The program that runs your Spark code. It decides what work exists, hands the pieces out, and collects the answers. There is exactly one per job, and when it stops the job is over.
executor	A process that does pieces of work for one driver. It is started for a job and exits with it. An executor holds a number of cores, and runs one piece of work per core at a time.
Spark master	The process that hands out the cluster's cores. It runs nothing itself. Drivers ask it for resources, it decides which machines will supply them, and it keeps a list of who is available. Not to be confused with the Kubernetes control plane, which hands out pods; this one hands out cores, and it is a pod itself.
Spark worker	A process that offers cores and memory to a master, and starts executors when told to. It announces itself to the master on startup, which is the line step 4 looks for. A worker is long-lived; the executors it starts are not.
standalone mode	Spark running its own master and workers, rather than asking another system for resources. It is Spark's simplest arrangement and the one deployed here. It exists because Spark predates Kubernetes and had to schedule for itself.
Service	A stable name for whichever pods currently carry a label. The one new Kubernetes object this round adds. Pods are replaced and each replacement gets a new address, so nothing can be configured to talk to a pod directly. A Service is the name that does not change: workers are told <code>spark://spark-master:7077</code> once and that stays true across every replacement.
cluster DNS	The name server every pod is given, which answers for Service names. It is what turns <code>spark-master</code> into an address inside the cluster. It answers for pods that are behind a Service, and for nothing else , which is the trap panel 9 describes.

3 • PUT THE SPARK IMAGE INTO THE CLUSTER'S IMAGE STORE • STEP 2

`µLab 1` showed that the cluster keeps its own images. That was a demonstration; here it is the obstacle. The Spark image is on your laptop and the cluster cannot see it, so the pods in step 3 would never start. Left to itself the cluster would download its own copy, 535 MB from Docker Hub, sixteen times over one classroom connection. Handing it the copy you already have takes well under a minute and no network at all.

2 ☐ Check the image is on the laptop, then hand it to the cluster.

Both lines are typed on your laptop, not in the `kubectl` shell.

```
docker image ls spark:3.5.4-python3
```

One row means the image is on this laptop:

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
spark	3.5.4-python3	5908cada5243	21 months ago	982MB

No rows at all means the pull in the announcement was missed. Run `docker pull spark:3.5.4-python3` now, and say so out loud: it is 535 MB, and the Lab Master needs to know which machines are downloading.

Now copy it across.

```
docker save spark:3.5.4-python3 | docker exec -i k8s ctr -n k8s.io images import -
```

<code>docker save</code>	Write an image out as a plain stream of bytes. With no <code>-o</code> file it goes to standard output.
<code>spark:3.5.4-python3</code>	Which image to write. <code>NAME</code> then <code>:</code> then <code>TAG</code> , the same spelling as everywhere else.
<code> </code>	Ordinary shell, nothing to do with Docker: send the left command's output straight into the right command's input, without ever writing a 1 GB file to disk.
<code>docker exec -i k8s</code>	Run a command inside the already-running <code>k8s</code> container. <code>-i</code> keeps its input open, which is what the pipe needs; there is no <code>-t</code> because nothing here is typed by hand.
<code>ctr</code>	The cluster's own low-level image tool, and the reason this has to be run in there: <code>ctr</code> exists only inside that container.
<code>-n k8s.io</code>	Which namespace of the store to write into. Kubernetes reads only <code>k8s.io</code> , and an image imported anywhere else is invisible to it, so the pods still will not start.
<code>images import</code>	Read an image stream and add it to the store.
<code>-</code>	Read from standard input rather than from a file. This is the other end of the pipe.

It prints one line per layer and then a total. Time it: on the teaching machine it took **37 seconds**, and a laptop will be slower. Then check the cluster's own list.

```
docker exec k8s crictl images
```

IMAGE	TAG	IMAGE ID	SIZE
docker.io/library/spark	3.5.4-python3	5908cada5243a	995MB

Write down the `IMAGE ID` from each of the two listings. They are the same image and they say so, which is exactly why the two stores are easy to confuse.

laptop ID _____ cluster ID _____

seconds the import took _____ cluster SIZE _____

4 · START THE MASTER AND TWO WORKERS · STEP 3

Spark's own cluster is a master that hands out cores and workers that supply them. Both are ordinary programs, so on Kubernetes both are ordinary Deployments, and the file below asks for exactly that. Nothing in it is new except the Service.

3 ☐ Apply the file, then watch the pods appear.

Typed in the `kubectl` shell from step 1, where the folder is mounted at `/lab`. With `kubectl` installed on your laptop instead, drop the `/lab/` and run it from the repository folder.

```
kubectl apply -f /lab/spark-standalone.yaml
kubectl get pods
```

<code>apply</code>	Make the cluster match what the file describes: create what is missing, change what differs, leave the rest. Running it twice is harmless.
<code>-f /lab/spark-standalone.yaml</code>	Read the description from this file. <code>-f</code> is for <i>file</i> . The path is the one inside the client container , not on your laptop.

Three objects are created, and it says so:

```
service/spark-master created
deployment.apps/spark-master created
deployment.apps/spark-worker created
```

What the file asks for, in three parts.

```
kind: Service          name: spark-master
  clusterIP: None      # resolve the name to the pod itself
  selector: { app: spark-master } # whichever pod carries this label
  ports: 7077 (submit), 8080 (web)

kind: Deployment       name: spark-master   replicas: 1
  image: spark:3.5.4-python3
  imagePullPolicy: IfNotPresent # use the store, never download
  command: spark-class org.apache.spark.deploy.master.Master --host spark-master
  volumeMounts: /lab (hostPath /lab, read-only)

kind: Deployment       name: spark-worker   replicas: 2
  command: spark-class org.apache.spark.deploy.worker.Worker \
    spark://spark-master:7077 --cores 1 --memory 1g
  volumeMounts: /lab (hostPath /lab, read-only)
```

<code>clusterIP: None</code>	Do not give the Service an address of its own; make the name answer with the pod's address. The master has to bind to the address its own name gives, and it cannot bind to a Service's.
<code>selector</code>	The label a pod must carry to be behind this Service. Not a pod name: that is the whole point, since pod names change.
<code>imagePullPolicy: IfNotPresent</code>	Use the store if the image is there, and only download if it is not. Without step 2 this is what would trigger the 535 MB download.
<code>--host spark-master</code>	Tells the master which name to publish itself under. It must match what the workers dial, or they connect and are then told to go somewhere they cannot reach.
<code>--cores 1 --memory 1g</code>	What one worker offers. Deliberately small, so the arithmetic is legible: the cluster has as many cores as it has worker pods.
<code>hostPath /lab</code>	Give the pod the folder the cluster container sees at <code>/lab</code> , which step 1 attached. Both Deployments get it: in <code>μLab 3</code> the master runs the driver, which reads the script, and the workers run the executors, which read the corpus.

Run `kubectl get pods` until all three read `1/1 Running`. It took about twelve seconds on the teaching machine. Write down the three names.

master pod _____	READY _____
worker pod _____	READY _____
worker pod _____	READY _____

5 • READ THE MASTER'S LOG AND FIND THE WORKERS • STEP 4

Three pods running proves Kubernetes started three programs. It does not prove they found each other, and they are two separate things: Kubernetes knows nothing about Spark, and would report a worker shouting into the void as perfectly healthy. The master writes down every worker that reports in, so its log is where the answer is.

4 ☐ Read what the master has logged since it started.

In the `kubectl` shell, `deploy/spark-master` means *the pod of that Deployment*, so the pod's name does not have to be typed.

```
kubectl logs deploy/spark-master | grep Master:
```

<code>logs</code>	Print what a container has written to its output since it started. Nothing is running or changed by this.
<code>deploy/spark-master</code>	KIND / NAME. Given a Deployment, <code>kubectl</code> picks one of its pods and says which.
<code> grep Master:</code>	Ordinary shell again: keep only the lines the master class wrote, and drop several hundred lines of startup from every other part of Spark.

The three lines that matter, as observed:

```
INFO Master: Starting Spark master at spark://spark-master:7077
INFO Master: Registering worker 10.42.0.5:33537 with 1 cores, 1024.0 MiB RAM
INFO Master: Registering worker 10.42.0.7:42663 with 1 cores, 1024.0 MiB RAM
```

The address in the first line is the Service name from panel 2, not a pod. The other two are the workers announcing themselves, one line each, with what they are offering. Your addresses will differ; the shape will not.

workers registered _____ cores each _____
cores in total _____ RAM each _____

Now answer the question this round is about.

Two numbers now agree: the worker Deployment was asked for `replicas: 2` and the master reports two workers. They are counted by different things and they are not the same number. Say which one is which, and which of the two would be wrong first if a worker pod were deleted. Panel 2 defines both counters: what a **Spark master** keeps a list of, and what a **Service** stands in for. μ Lab 1 panel 9 is the other half, where a controller was shown replacing a deleted pod.

Solved means panels 1 to 5 are done: the cluster re-created with the lab folder attached, the Spark image listed by `crictl images`, master and two workers all reading 1/1 Running, and the master's log naming both workers with the cores each one offered.

Reached this line? Then you have solved the challenge.



Referee: tick the box on the left now, and write the time. Do not wait for the caucus. Then carry on below — the extensions are not optional, they are the rest of the round.

Solved at _____ Referee's initials _____

6 • RUN A JOB ON THE CLUSTER YOU JUST BUILT

Everything so far shows Spark is **running**. This shows it **computes**. The job is the one that ships inside the image: throw darts at a square, count how many land inside a circle, and multiply. It needs no data, so nothing has to be copied anywhere, which makes it the cheapest possible proof that the master, the workers and a driver can all reach each other.

Typed in the `kubectl` shell. The driver runs **inside the master pod**, which looks like a detail and is not: panel 9 says what happens when it runs anywhere else.

```
kubectl exec deploy/spark-master -- /opt/spark/bin/spark-submit \
--master spark://spark-master:7077 \
--class org.apache.spark.examples.SparkPi \
local:///opt/spark/examples/jars/spark-examples_2.12-3.5.4.jar 100
```

<code>exec</code>	Run a command inside a container that is already running, the same idea as <code>docker exec</code> in step 2, one level in.
<code>--</code>	End of <code>kubectl</code> 's own arguments. Everything after it belongs to the command being run, so <code>--master</code> is read by Spark and not by <code>kubectl</code> .
<code>spark-submit</code>	Spark's launcher: start a driver, ask a master for cores, run the code, print the result, exit.
<code>--master spark://...:7077</code>	Which master to ask. The Service name from panel 2 and port 7077, which is the one the file labelled <i>submit</i> .
<code>--class org.apache...SparkPi</code>	Which program inside the jar to run. A jar holds many; this names one.
<code>local:///opt/spark/...jar</code>	Where the jar is. <code>local:</code> means <i>already present in the image on every machine</i> , so Spark ships nothing. The third slash is the root of that filesystem.

100

The program's own argument: how many slices to split the darts into. More slices, more pieces of work, and a more precise answer.

Several hundred lines scroll past and one line matters. On the teaching machine the whole thing took **7 seconds**:

Pi is roughly 3.1411035141103514

The digits after the third will not match anyone else's, including a second run of your own, because the darts are thrown at random. Anything near 3.14 is a success.

your value _____ seconds _____

7 • DELETE A WORKER POD AND WATCH THE MASTER NOTICE

µLab 1 deleted a pod and watched a controller put it back. The same thing happens here, except that something else was relying on that pod, so there are now two reactions to watch instead of one, written down by two different programs.

```
kubectl get pods
kubectl delete pod <one of the two workers>
kubectl get pods
kubectl logs deploy/spark-master | grep Master: | tail -3
```

```
delete pod Remove one pod. It removes nothing else: the Deployment that asked for it is untouched and still wants two.
| tail -3 Shell: keep the last three lines, which are now the three most recent things the master wrote.
```

As observed, in this order:

```
INFO Master: Removing worker worker-...-10.42.0.5-33537 on 10.42.0.5:33537
INFO Master: Telling app of lost worker: worker-...-10.42.0.5-33537
INFO Master: Registering worker 10.42.0.9:40593 with 1 cores, 1024.0 MiB RAM
```

Write down the new pod's name and the new address in the master's log. Then say which of the two put the worker back, the Kubernetes controller or the Spark master, and what the other one did instead. Panel 2 defines what each keeps track of, and the three lines above are in the order the two of them acted.

8 • EXTRA MILE • GIVE THE CLUSTER MORE WORKERS THAN THE LAPTOP HAS CORES

Each worker was told `--cores 1` in step 3, so the cluster's core count is its worker count. Nothing stops you asking for more workers than the laptop has cores. Predict what the master will report before you run it, then find out.

```
kubectl scale deployment spark-worker --replicas=8
kubectl get pods
kubectl logs deploy/spark-master | grep -c "Registering worker"
```

```
scale ... --replicas=8 Edit one field of a Deployment that already exists. It creates nothing and starts nothing by hand.
grep -c Count matching lines instead of printing them. Note it counts every registration since the master started, including the ones from step 4 and panel 7.
```

cores you predicted _____ cores reported _____

cores the laptop has _____ pods Running _____

Then run panel 6 again and time it. Eight workers of one core each is eight cores as far as the master is concerned. Say whether the laptop agrees, and what the master would have to be told in order to find out.

9 · ANNEX · WHY THE DRIVER HAS TO RUN SOMEWHERE THAT HAS A NAME

Panel 6 submitted the job from inside the master pod. Submitting from a pod of your own looks tidier and does not work. Executors are told to call the driver back by name, and cluster DNS answers only for pods that are behind a Service. A pod started on its own has no name anyone can resolve, so every executor exits immediately and the driver waits for cores that never arrive, printing this every fifteen seconds:

```
WARN TaskSchedulerImpl: Initial job has not accepted any resources; check your
cluster UI to ensure that workers are registered and have sufficient resources
```

The message blames the workers, and the workers are fine. The master pod already has a Service in front of it, which is the whole reason panel 6 borrows it.

What production does instead. Standalone mode is Spark scheduling for itself, from before Kubernetes existed. Given a real cluster, a submission made with `--master k8s://...` skips all of this: Kubernetes is asked directly for a driver pod and executor pods, they exist only while the job runs, and there is no master, no workers and nothing standing idle in between. It is also several more things to get right, which is why it is here and not in step 3.

Done when the referee can say, without reading this sheet, why the cluster could not start the Spark image until it was handed a copy, and what a Service is for.